

Multi-Agent Planning under Partial Observability: A POMDP Approach via POMCP

Muhammad Zahid

Department of Artificial Intelligence
Manchester Metropolitan University
Manchester, United Kingdom
25925501@stu.mmu.ac.uk

Abstract

This paper presents a decentralized approach for multi-agent decision-making in partially observable environment. **Partially Observable Markov Decision Processes (POMDPs)** are a general model for planning and solving problems under uncertainty. POMDPs basically depend on particle-based local beliefs and reward shaping. The proposed allows for a scalable online planner in uncertain grid-like environments without explicit inter-agent communication. The **Partially Observable Monte Carlo Planning (POMCP)** model provides a robust estimation for online planning. The system handles large state spaces by simulating only a few sampled belief particles rather than listing the full belief space, can handle partial observability and randomness of action, and the flexible reward shaping encourages exploration and goal-directed behavior.

Introduction

Autonomous multi-agent systems operate consistently in noisy environments, where observations received from sensors are often incomplete, distorted and hidden. Such uncertainty in observations leads to partial observability, such conditions are common challenges in applications such as robot navigation, drone swarms, surveillance, search and rescue missions and in areas such as traffic navigation.

Partially observable conditions are common challenges in applications such as robot navigation, drone swarms, and search and rescue. Partially Observable Markov Decision Processes (POMDPs) are a general model for planning problems under uncertainty, but computing the exact solution scales poorly in practice. However, computing exact solutions scale poorly for POMDPs in practice due to the number of possible past action–observation histories exploding and the belief space growing with state dimensionality.

To solve this problem many algorithms have emerged. Among them are online algorithms, Monte Carlo planning which can adapt to the environmental changes and greatly reduce computational costs of solving POMDPs.

One popular algorithm is POMCP. To balance exploration and exploitation, it adopts the **Upper Confidence bounds applied to Trees (UCT)** algorithm for action selection, this

enables scalable planning without the need to explicitly maintain the full belief space.

This work investigates a decentralized multi-agent navigation framework based on POMDP modeling and POMCP-based online planning. Each agent maintains an independent belief state and plans locally without any inter-agent communication.

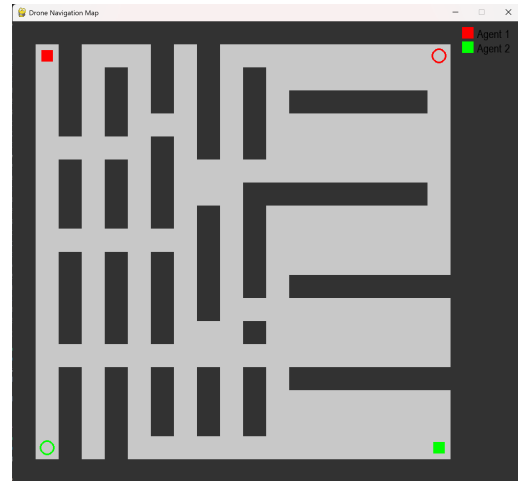


Figure 1: Grid MAP

The proposed approach is evaluated in a partially observable grid-based environment shown in Fig. 1, and system performance is analyzed in terms of reward, efficiency, and goal achievement.

Implementation of the code is available on.¹

Related Work

POMDP Foundations and Multi-Agent Limits

POMDPs provide a general framework for single-agent decision-making under uncertainty, but exact solutions are impractical due to the curse of history and curse of dimensionality. Extending this to multi-agent settings gives Dec-POMDPs, where joint spaces grow rapidly, requiring

approximate methods that exploit problem structure to reduce complexity (Mao et al. 2023).

Decentralized POMDP approximations

To tackle the intractability of Dec-POMDPs practical approaches to Dec-POMDPs focus on computationally manageable estimations. Key methods include:

- **Common Information Reductions:** These techniques transform the decentralized problem into a centralized POMDP solved by a virtual coordinator. Feasibility relies on maintaining a compact representation of shared information and assuming agents have finite local memory Mao et al. (2023).
- **Finite-Memory Controllers:** This approach forces agent policies to depend on a limited internal state or a short history window. This deliberate restriction allows practical learning and planning while limiting the approximation error Mao et al. (2023).
- **Policy Graph Methods:** These algorithms iteratively optimize compact, fixed-size policy graphs (or controllers). A significant advantage of this approach is its flexible nature, allowing it to adapt to complex objectives like non-linear information rewards Lauri et al. (2020).

Monte Carlo Tree Search and POMCP variants

POMCP gives us Monte Carlo Tree Search (MCTS) integrated into POMDPs, enabling us to perform online planning without explicit belief updates. By sampling states from particle-based beliefs, using UCT for action selection, and performing rollouts to estimate expected returns, POMCP provides a balance between exploring potential new actions and exploiting high-value options, providing applications in large and partially observable domains.

For decentralization in multi-agent domains, common strategies include:

- building a tree centered around the agent and predicting other agent’s actions during simulation.
- constructing a joint-action tree and marginalizing actions local moves selection.

Particle-Based Belief Handling and Local Approximation

Particle filtering approximates beliefs in large state spaces, allowing agents to perform POMCP simulations without knowing the full state. Each agent maintains a local belief set, and subjective approximations model local states and nearby tasks while assuming probable locations of other agents, further reducing computational cost (Amini et al. 2023). Trajectory-based classifiers can judge team behavior from past observations, though they require offline training, trading online scalability for predictive accuracy (Ribeiro et al. 2024).

Reward Design and Exploration Strategies

Reward shaping is essential in multi-agent planning. To preserve optimality while guiding agents, we use Potential-

Based Reward Shaping (PBRs) based on Manhattan distance to the goal. Penalties like: time penalties, wall collisions, and re-visits discourage inefficiency. Information gains, is a reduction in belief entropy, encourage exploration of unobserved areas and penalize redundant revisits. These reward components are combined to guide the agent both toward task completion and effective exploration, mirroring the reward function implemented in the State Manager.

Decentralized Multi-Agent Planning and Execution

Building on recent research, the current decentralized framework, uses independent POMCP simulations per agent without inter-agent communication. Each agent maintains its own belief and action-observation history. A Multi-Agent Controller synchronizes joint action execution and re-roots all agent’s search trees after new observations. This design, implemented in POMCP, State Manager, and belief modules, maintains scalability, handles partial observability, and local uncertainty.

Approach

The proposed method integrates fully decentralized POMCP planning Amini et al. (2023), particle-based local beliefs, and targeted reward shaping for multi-agent navigation in partially observable grid worlds. It builds on POMCP’s particle-based UCT rollouts, navigates the single action tree, and adopts reward and update strategies Wang et al. (2023). Because the agent’s guess about the world, its choices and the rewards it obtains are based only on what it can observe, our system approach enables scalable and robust multi-agent decision-making under uncertainty. This approach is implemented in the POMCP framework and Multi-Agent Controller.

Problem Formulation

The multi-agent navigation task is formulated as a Partially Observable Markov Decision Process (POMDP), defined by the tuple $\langle S, \mathcal{A}, \mathcal{O}, T, Z, R, \gamma \rangle$.

State Space S

The state space represents the true underlying configuration of the environment $S = \{s_1, s_2, \dots, s_n\}$, including agent positions, goal, and obstacle locations in a two-dimensional grid world. The true state is not directly observable by the agents.

Action Space $\mathcal{A}$

Each agent has a discrete action set $\mathcal{A} = \{up, down, left, right, stay\}$, allowing movement within the grid environment.

Observation Space $\mathcal{O}$

Agents receive partial observations $\mathcal{O} = \{o_1, o_2, \dots, o_k\}$ within a limited sensing radius. Observations provide local information about free and occupied cells, while the remainder of the environment remains unobserved.

Observation Model Z

Probability of receiving observation o after executing action a and reaching state s' .

$$Z(o | s', a) = P(o | s', a) \quad (1)$$

The observation received depends on the agents location, sensing range, and the environment.

Transition Model T

It is the probability that environment transitions from state (s) to state (s') after action (a) .

$$T(s' | s, a) = P(s' | s, a) \quad (2)$$

State transitions are deterministic and dependent on the selected action and environmental constraints such as obstacles.

Reward Function R

$$R = R_{\text{env}} + R_{\text{pbrs}} + R_{\text{info}} \quad (3)$$

The reward function R encourages goal-directed behavior, providing positive reward for reaching the goal, penalties for collisions or invalid actions, revisiting previously explored cells, and small step costs to discourage inefficient exploration.

$$R_{\text{env}} = -c_{\text{step}} + R_{\text{goal}} + R_{\text{wall}} + R_{\text{revisit}} + R_{\text{stay}} \quad (4)$$

Potential-Based Reward Shaping (PBRs) is used to guide agents toward the goal.

$$R_{\text{pbrs}} = \gamma \cdot \Phi(s') - \Phi(s) \quad (5)$$

The potential function is defined as the negative inverse of the Manhattan distance to the goal, providing smoother reward gradients during planning.

$$\Phi(s) = -\frac{1}{1 + d(s, g)} \quad (6)$$

where $d(s, g)$ is Manhattan distance to the goal. Information gain reward is integrated to encourage exploration under uncertainty.

$$R_{\text{info}} = \text{clip}(H(b) - H(b'), -1, 1) \quad (7)$$

The reward equals the decrease in belief entropy between consecutive belief states, encouraging actions that lower uncertainty about the environment; for numerical stability the reward is clipped to a fixed range. where:

$$H(b) = -\sum_i b_i \log b_i$$

is the entropy of a belief distribution b measuring uncertainty. Here b_i is probability assigned to the i -th possible state. For a belief over a grid cell being occupied or free.

The combined reward function enables effective planning under partial observability and maximizes goal achievement, path efficiency, collision avoidance, belief-driven exploration, and information acquisition.

Discount Factor γ

Balance between how much immediate versus Future rewards are valued.

- $\gamma \approx 1$: long-term planning
- $\gamma < 1$: short-term preference

Objective Function

Each agent aims to maximize its expected cumulative discounted reward over a finite horizon, while planning independently under partial observability. Planning is performed over a finite horizon H , with future rewards discounted by γ . The agent's goal is to find a policy π that maximizes the expected cumulative discounted reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t, s_{t+1}) \right] \quad (8)$$

Each agent's policy $\pi(a | h)$ is conditioned on its local action-observation history $h = (a_0, o_0, \dots, a_{t-1}, o_{t-1})$ rather than full state. The full state $s \in \{S\}$ contains the complete positions of all agents and the true configuration of the environment.

System Architecture

This overall system is designed as a decentralized **POMDP** framework where each agent maintains its own belief state, performs local planning using Partially Observable Monte Carlo Planning (**POMCP**) and updates local action-observation based on the planned outcome. The architecture integrates four main components:

- **Environment and State Management**
- **Belief State Management**
- **Local POMCP-Based Planning**
- **Multi-Agent Execution**

This design ensures modularity, scalability, and robustness under partial observability, uncertainty and independent planning.

Architectural Overview

Using a decentralized approach, each agent manages its own belief state and runs its own planning process without any communication with other agents. The system environment takes all agent's chosen action as joint actions and executes them providing each agent with its own reward and observations. This setup avoids the high computational cost of centralized planning. The Fig. 2 shows the Execution cycle and Data Flow in Decentralized POMCP Planning.

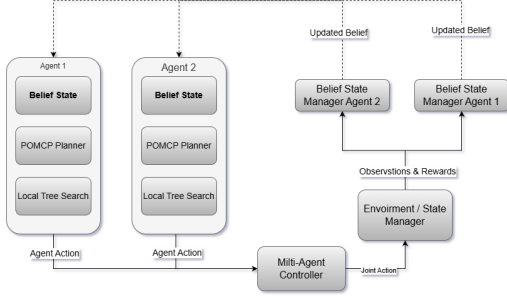


Figure 2: Execution Cycle and Data-Flow

Agents plan their next action at each step based on their belief state. The environment then executes all selected actions at the same time. The environment then returns the observations which are used to update each agent's belief and improve future planning.

Environment and State Management

The State Manager is responsible for maintaining the true state of the environment and executing agent actions. Since the modeled environment is a grid world containing obstacles, goal and start positions of the agents, the environment has access to the real map while the agents do not observe the complete map directly.

State transitions are deterministic with respect to agent actions, depending on the boundary and obstacle constraints. Invalid actions, such as attempting to move into obstacles or outside the grid, result in the agent remaining in its current position. Observations are local and limited in range, but the agents have partial observability where they can see the real map near them, while the rest is represented through their belief map.

Belief State Representation and Management

For partial observability, each agent maintains an independent belief state of the environment. The Belief State Manager maintains this belief as a probabilistic grid, where every cell is encoded with possibility of being free or occupied. This representation allows agents to think, and act based on the uncertainty of the unobserved cells.

Particle-Based Belief update: Practically belief is represented as

$$b(s) \approx \frac{1}{N} \sum_{i=1}^N \delta_{s_i}$$

here each particle is represented as s_i which is sampled map configuration. After an action a and receiving observation o particles are weighted

$$w_i \propto O(o | s_i, a)$$

and then resampled to the updated belief $b'(s)$. These particle-based approximation allow the POMCP to use sampled worlds instances during simulation process.

Soft Belief Updates and Uncertainty Propagation: To manage uncertainty, observations update beliefs to softened values (e.g., 0.1 or 0.9) rather than absolute 0 or 1. Unobserved cells maintain their previous probability close to 0.5 Fig. 3.

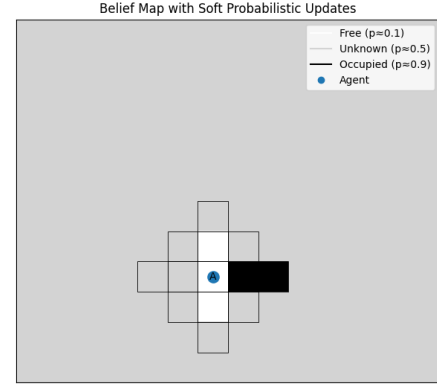


Figure 3: Agent Belief Map

Thus, uncertainty is maintained in two ways:

- soft probability (0.1/0.9) are used instead of absolute values.
- by leaving unobserved cells with the previous probability

Hence, we can say:

$$b_{t+1}(x, y) = \begin{cases} 0.1 & \text{if the cell is observed as free} \\ 0.9 & \text{if the cell is observed as occupied} \\ 0.5 & \text{otherwise if unobserved} \end{cases}$$

To enable planning under uncertainty, the continuous belief distribution is approximated through particle sampling. Multiple environmental maps drawn from this belief, represent a possible world state. These sampled maps are used by the POMCP planner during simulations, allowing effective decision-making under partial observability without requiring exact probabilistic inference.

Local POMCP-Based Planning

POMCP is a general-purpose planner for large-scale decision-making under uncertainty. In this multi-agent navigation scenario, POMCP enables agents to make decisions by simulating possible futures based on their local belief.

Each agent uses an independent POMCP planner to select actions by performing Monte Carlo Tree Search over action-observation histories.

Using sampled maps from its belief, each agent simulates action sequences up to a fixed horizon. Actions are scored with discounted rewards, balancing exploration and exploitation. Local planning with independent tree ensures computational efficiency.

Fig. 4 is a basic conceptual idea of how a POMCP Tree actually performs Action Selection using UCB.

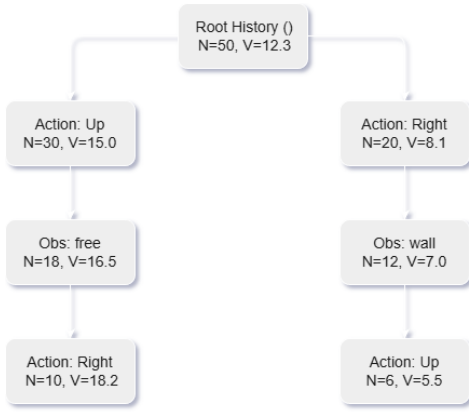


Figure 4: POMCP Tree

POMCP combines **MCTS** with belief-based simulations. Each agent builds its search Tree separately, where nodes are histories of actions and observations. The planning works as follows:

- **Action Selection:** At each node, the agent selects an action using an **Upper Confidence Bound (UCB)** strategy. This allows the agent to explore less preferred actions, creating a balanced approach between exploration and exploitation.

$$UCB(N, n, V) = \begin{cases} +\infty, & n = 0 \\ V + c\sqrt{\frac{\ln N}{n}}, & n > 0 \end{cases}$$

where N is the total visits to the parent node, n is visits to the action node, V is the estimated value, and c is the exploration constant.

- **State Transition:** Each agent simulates the selected action on the sampled map particle received from the belief state manager and generates the next state based on the transition model.
- **Observation:** A local Observation is made for the next state. Belief is updated inside the simulation to match this hypothetical observation.
- **Reward Calculation:** Each of these action and observation pair is carefully given a calculated reward based on the combination of environment progress, reward shaping, and information gain.
- **Rollout:** On reaching a node that has yet to be expanded or reaching the max depth (horizon), rollout policy is used, which quickly estimates the future reward without further expansion using a greedy heuristic toward the goal.
- **Backprop:** After each simulation the estimated total reward value is passed back through all the nodes updating the visit count and adjusting its estimated value in reflection of the new result. Through this the planner learns which actions are better through many simulation trials.

After repeated simulations, the agent selects the action with the highest visit count or value.

Search Tree Structure

The planning process relies heavily on the search tree made of history nodes, action nodes, and observation nodes. Each node maintains its own visit counts and value estimates learned during simulations. This follows the main ideas of POMCP and lets the planner reuse what it already knows. After the action is executed in the real environment and an observation is received, the planner re-roots its search tree at the corresponding history node. This discards branches that are no longer consistent with the observed outcomes, improving planning efficiency over time and keeping relevant parts of the tree.

Multi-Agent Execution

The Multi-Agent Controller manages the overall execution for all agents. At each step, it calls each agent's local planner to get an action. These actions are then combined into one joint action and applied to the environment simultaneously.

After the actions are applied, the controller gathers the observations and rewards information, updates each agent's belief state, and logs their trajectories.

Agents plan independently using their own beliefs and observations. A Multi-Agent Controller integrates their chosen actions into a joint action for the environment. Agents then update their beliefs and re-root their search trees based on the new observations. This decentralized approach ensures scalability under uncertainty.

Execution Cycle: The execution loops at every step is as follows:

1. Each agent independently performs local planning using its POMCP planner.
2. The selected actions are combined into joint actions.
3. The environment executes joint action and updates the true state of all agents.
4. Each agent receives an individual observation and reward for their action.
5. Belief states are updated based on the new observations obtained.
6. Planning trees are re-rooted to match the updated histories.

This cycle is repeated until all agents reach their goals.

Design Rationale

The chosen design prioritizes scalability, modularity, and robustness under uncertainty. By separating belief management, planning, and execution, the system remains easy to extend and understand. The decentralized planning strategy helps with computational effectiveness of joint action spaces, while belief-driven POMCP planning supports effective decision making in a partially observable environment.

Reward Design and Shaping

This section plays a crucial role in the effectiveness of the system and the ability of the agents to navigate towards the goal. In our system, rewards guide agents to achieve

goals efficiently while exploring the environment. The reward function consists of three main components:

Environment Reward

- **Goal Reward:** a large positive reward is given when an agent reaches its goal.
- **Time cost and inefficient moves:** small negative penalties for staying still, hitting walls, or revisiting cells.
- **Moving closer to the goal** gives a small reward, while running into obstacles gives a penalty.

Potential-Based Reward Shaping (PBRs)

- Helps the agent make progress toward the goal without changing the optimal policy.
- It assigns a potential value based on the Manhattan distance to the goal.
- Formula:

$$r_{PBRs} = \gamma \cdot \Phi(new) - \Phi(old) \quad (9)$$

Where Φ is a function of distance to the goal, and γ is a discount factor.

- This encourages the agent to move toward the goal gradually.

Information Gain Reward

- Encourages exploration in case of uncertainty about the environment.
- Based on belief entropy: rewards actions that reduce uncertainty in the agent's belief of the environment.
- Observing new areas gives a positive reward, while revisiting known areas gives negative a reward.

Experimental Setup

Environment

The environment for our multi-agent setup is a 2D grid world, represented by zeros and ones. The map is stored in a .xlsx file and accessed through a data loading program, forming a 20×20 grid. Obstacles are stationary, and the map has a mazelike structure with long corridors through which agents must navigate. Start and goal positions are hard-coded into the map, so the number of agents is fixed and can only be changed manually.

Obstacles, goals, start positions, and free spaces are encoded as 1, 3, 2, and 0, respectively. Agent's movements are updated in real time, providing a live visualization of their behavior. Each episode continues until all agents reach their goal positions or a maximum number of steps is reached.

Configuration and Planning Parameters

In our system, for simplicity we deploy a fixed number of agents: 2. Each agent is allowed five actions; they can move **up**, **down**, **left**, **right**, and **stay**:

- **Actions:** {up, down, left, right, stay}.

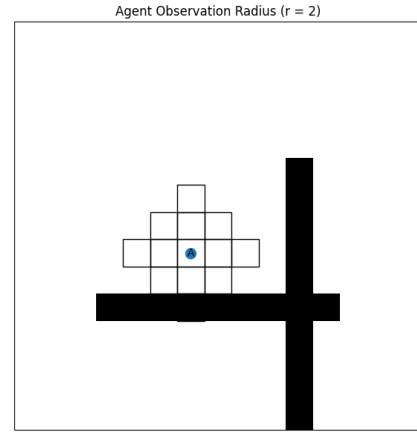


Figure 5: Agent observation Area

The agents are allowed to perceive the real environment up till a limited radius of 2 at any point, while the rest of the map is hidden for the agent (see Fig. 5).

Agent's local observation region with Manhattan radius $r = 2$. The agent observes true cell states within the high-lighted area while unobserved regions remain uncertain.

All the agents are collectively allowed only 500 max steps per episode.

Planning is performed using **POMCP**. The planning horizon (max depth) is limited to 3, and the number of simulations per decision step is limited to 22. The reward discount factor γ is set differently in two places:

- potential-based reward shaping factor in the agent state manager:

$$\gamma_{\text{shape}} = 0.90$$

- standard discount factor used in POMCP:

$$\gamma_{\text{POMCP}} = 0.99.$$

This reflects the trade-off between immediate reward and future reward.

This configuration was chosen through trial and error to balance computational efficiency and the effectiveness of agent decision making.

Execution

Running the program triggers it to load the Grid map, upon successfully loading the map, our program uses a **Pygame** library for visualization of the map.

Execution loop:

- The simulation then proceeds in discrete time steps and at every step the agent runs its own POMCP planner independently based on the agent's belief, their own Tree search (MCT), and separate action-observation history. Representation of this is shown in Fig. 6

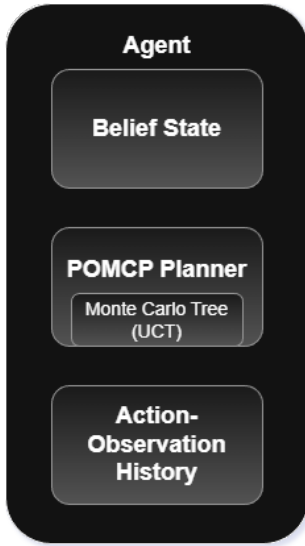


Figure 6: Agent Internal Pipeline

- During the planning phase, each POMCP iteration samples a possible world state from the agent's belief state manager. Then proceeds to perform a simulated **Rollout** using **UCB** based action selection, based on the simulated results (observations and rewards) beliefs are updated.
- After planning each agent then selects its local actions, from the root of the search tree the most optimal action is opted. The independent actions are then merged into joint actions by Multi-Agent Controller which simultaneously executes them in the real environment.
- Based on the received local observations, each agent updates its belief state based on the observed information, extends its local action-observation history, and then re-roots its POMCP tree to retain only relevant branches.

This entire process repeats at each step until termination condition is achieved.

Evaluation Metrics

Agent's performance is measured mainly through cumulative reward which is used to discern the performance.

Cumulative reward: The total reward the agent accumulates upon completion of the program. This metric of evaluation captures task efficiency, penalties for collision, revisiting previously explored states, forbidden actions, time cost information gathering behavior all implemented through reward shaping. High reward represents more effective decision making with partial observability.

Path length: Although not used directly as an evaluation metric, the path agents choose can also be visually interpreted to identify the agent's behavior. Whether it shows extreme randomness, getting stuck in long corridors, wasteful exploration or oscillations.

Results and Analysis

Five independent episode runs were conducted to evaluate individual agents performance and the performance of the system as a whole. Agent 1 is marked as **Red** and Agent 2 is marked as **Green** as shown in Fig. 7. Table 1 provides the performance metrics for each episode, including the individual agents total reward, steps counts, and goal achievement status of both agents.

No	Status	Agent 1 Reward	Agent 2 Reward	Steps	Total Efficiency
1	goal	14,542	-76	296	48.86
2	goal	55	348	73	5.52
3	fail	1,485	-304	499	2.37
4	goal	15	6,486	131	49.62
5	goal	-26	12,813	186	68.73

Table 1: Agent Performance Comparison Across Episodes

It can be observed that four out of five episodes were a success, and both agents achieved their goal status while episode 3 remained a failure. Fig. 7 gives a visual idea of the reason behind agent 2 failure, it clearly can be seen the agent although shows goal directed behavior (getting close to the goal) it has a much higher tendency to roam around (exploratory behavior).

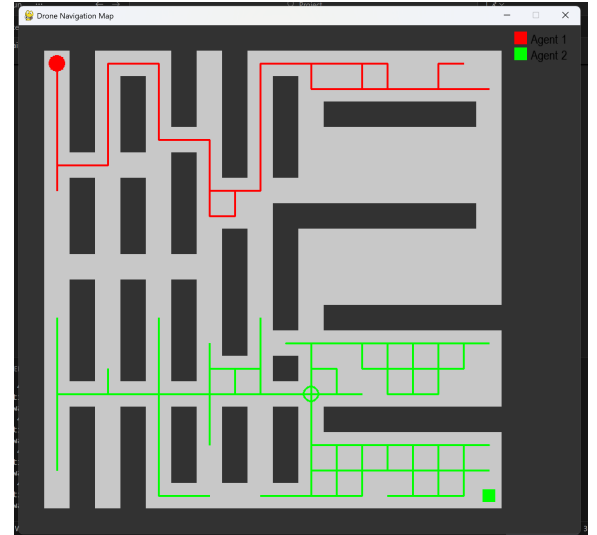


Figure 7: Episode 3

As can be seen in the Table. 1 the reward status of both agents shows significant variance, one agent out performs the other in every episode. Episode 1, 4 and 5 indicates remarkable system efficiency, both agent reaching their goal in just 296, 131, and 186 steps respectively, while episode 2, 3 show Low amount of efficiency. Episode three remains the only failure result requiring the maximum number of steps and significantly high exploratory behavior.

Keep in mind that episode success is evaluated at the system level; therefore, an individual agent may reach its goal even when the episode is classified as a failure overall.

Episode	Goal?	Reward	Steps	Efficiency
1	Yes	14,542	296	49.12
2	Yes	55	73	0.75
3	Yes	1,485	499	2.98
4	Yes	15	131	0.11
5	Yes	-26	186	-0.14
Avg	100%	3,214	197	10.5

Table 2: Agent 1 Performance Metrics

Tables 2 and 3 further emphasizes decentralized nature of the system, considering both agents are highly effected by local observations and belief matrix. The result further confirm this behavior as, Agent 1 shows an average low efficiency with high variance in reward values, while Agent 2 achieves higher average reward comparatively, but both also demonstrates variability across the five episodes .

Episode	Goal?	Reward	Steps	Efficiency
1	Yes	-76.22	296	-0.26
2	Yes	348.66	73	4.77
3	No	-304.70	499	-0.61
4	Yes	6,486.02	131	49.51
5	Yes	12,813.76	186	68.87
Avg	80-100%	3,854	197	24.5

Table 3: Agent 2 Performance Metrics

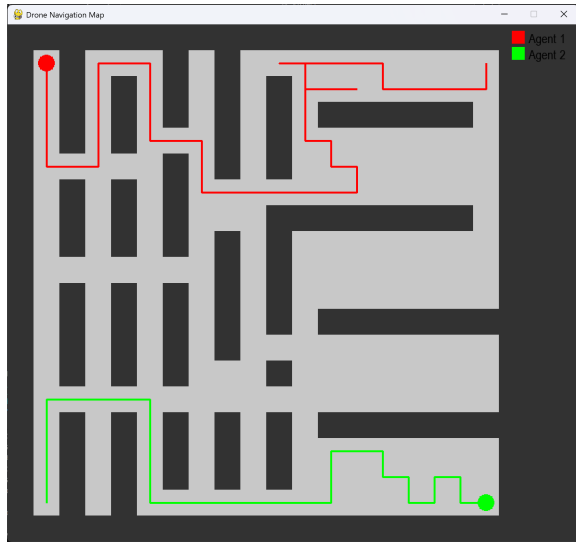


Figure 8: Final episode

As a final verdict Fig. 8 proves another successful Result of both agents reaching the goal and receiving a positive reward indicating the program works successfully. Agent 1 shows a reward of 57 and Agent 2 a reward of 3243, both agents reaching the goal in just 66 steps. But, both agents still show high level of variance between the to reward. al-

though this behavior proves the agents are working without any shared belief these results can be improved by using a more effective reward design and applying a more robust POMCP algorithm.

Discussion

The experimental results demonstrate that the POMDP-based framework for multi-agent navigation, implemented using Partially Observable Monte Carlo Planning (POMCP), enables agents to operate effectively under partial observability, while also showing variability in performance across episodes. Independent belief evolution and localized observations show the impact on the individual rewards and efficiencies. The asymmetric reward distribution between agents suggests that, due to local observations and independent belief updates, one agent may receive adequate reward information earlier than the other. Episodes with high system efficiency indicate a transition from exploratory behavior to persistent goal exploitation, reflecting faster belief convergence. In comparison episodes that demonstrate lower efficiency like Episode 3 exhibit prolonged exploratory behavior due to uncertainty in the agent's belief state.

Overall, the results confirm that the system can handle large state spaces and partial observability by simulating sampled belief particles, and that flexible reward shaping balances exploration and goal-directed behavior. The POMCP model thus provides a robust approximation for on-line planning in decentralized POMDPs. However, performance is sensitive to planning horizon, reward discounting, and simulation budget. The lack of agent coordination also suggests future improvements in system efficiency.

Conclusion

Each agent maintained independent belief states and planned locally without any inter-agent communication in an uncertain environment. Multiple experiments suggest that the system can achieve goal-directed behavior despite the limited observability. While most of the episodes were successful agents finding their goal, occasional failures and asymmetric agent performance highlight the challenges faced by uncertainty and independent exploration. Future work should explore improved reward shaping and adaptive planning in complex and ever-changing environments. Extensions to more complex environments and greater number of agent population for further assessing the scalability and robustness of the proposed approach.

References

- Amini, S., Palhang, M. & Mozayani, N. (2023), 'Pomcp-based decentralized spatial task allocation algorithms for partially observable environments', *Applied Intelligence* **53**(14), 12613–12631.
- Lauri, M., Pajarinen, J. & Peters, J. (2020), 'Multi-agent active information gathering in discrete and continuous-state decentralized pomdps by policy graph improvement', *Autonomous Agents and Multi-Agent Systems* **34**(42), 1–44.

- Mao, W., Zhang, K., Yang, Z. & Başar, T. (2023), Decentralized learning of finite-memory policies in dec-pomdps, in ‘IFAC-PapersOnLine’, Vol. 56, Elsevier, pp. 2601–2607.
- Ribeiro, J. G., Oren, Y., Sardinha, A., Spaan, M. & Melo, F. S. (2024), ‘Recbayes: Recurrent bayesian ad hoc teamwork in large partially observable domains’, *arXiv preprint arXiv:2506.15756* .
URL: <https://arxiv.org/abs/2506.15756>
- Wang, Q., Liu, F. & Luo, B. (2023), A partially observable monte carlo planning algorithm based on path modification, in ‘Proceedings of the Asian Conference on Machine Learning (ACML)’, Vol. 222, PMLR, pp. 1–16.
URL: <https://proceedings.mlr.press/v222/wang24c.html>